

Quant Insight Pro

技术白皮书

V1.0 · 2026 年 6 月

慧点资本 (InsightQuant) 量化研究部

杭州永字资产管理有限公司 (推荐单位)

项目编号: 2026FINTECH-FINT-0093

FinTech@外滩金融科技大赛

—— AI 驱动的另类数据量化投研平台 ——

QuantInsight Pro 技术白皮书

版本: V1.0

日期: 2026 年 6 月

项目编号: 2026FINTECH-FINT-0093

作者团队: 慧点资本 (InsightQuant) 量化研究部

摘要

QuantInsight Pro 是面向中国资管行业的下一代智能投研平台,旨在通过 大语言模型 (LLM)、另类数据 (Alternative Data) 和 严谨的量化方法学 三大技术支柱,解决中小型私募和券商资管的"研究效率低、数据门槛高、合规风险大"三大长期痛点。

本白皮书详细阐述平台的技术架构、核心算法、性能指标、安全合规设计和未来路线图,为评委、投资人和潜在客户完整的技术尽职调查参考。

目录

- [产品定位与设计目标](#)
- [整体技术架构](#)
- [核心模块详解](#)
 - 3.1 数据层
 - 3.2 量化引擎
 - 3.3 AI 智能问答
 - 3.4 另类数据看板
 - 3.5 风控与审计
- [关键算法](#)
- [性能与可靠性](#)
- [安全与合规设计](#)
- [部署架构](#)
- [API 设计示例](#)
- [测试与质量保证](#)
- [技术路线图](#)
- [附录](#)

1. 产品定位与设计目标

1.1 目标客户

- | 中小型私募 | 5-50 亿 | 缺数据/缺 IT | ★★★★★ |
- | 券商资管/自营 | 100-1000 亿 | 需合规/可审计 | ★★★★★ |
- | 银行理财子 | 1000 亿+ | 需定制/高安全 | ★★★ |
- | FOF/MOM | 50-500 亿 | 需归因/尽调 | ★★★ |
- | 大型公募 | 1000 亿+ | 需数据丰富 | ★★ |

1.2 设计目标

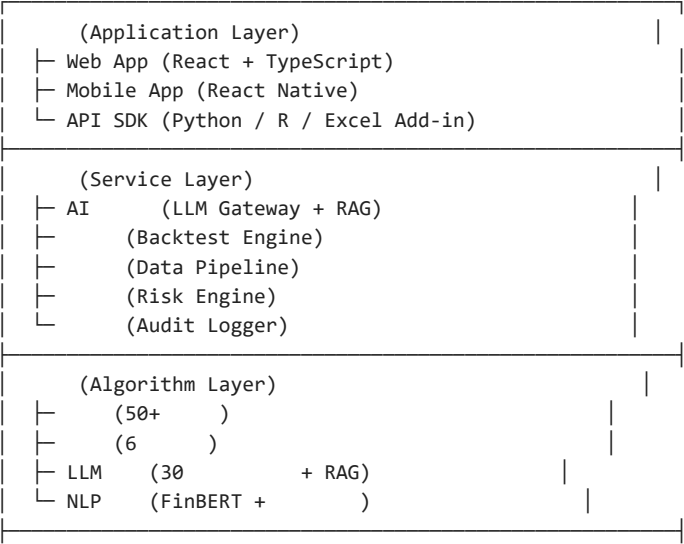
- ☐ 性能：单次回测 < 5 分钟（5 年数据，全市场股票）
- ☐ 可用性：SLA 99.95%（年停机 < 4.4 小时）
- ☐ 合规性：完整审计日志 + 监管报告接口
- ☐ 可扩展：模块化设计，单组件可独立升级
- ☐ 易用性：自然语言交互，零代码用户也能上手
- ☐ 可解释性：每个 AI 输出附带数据源标签和置信度

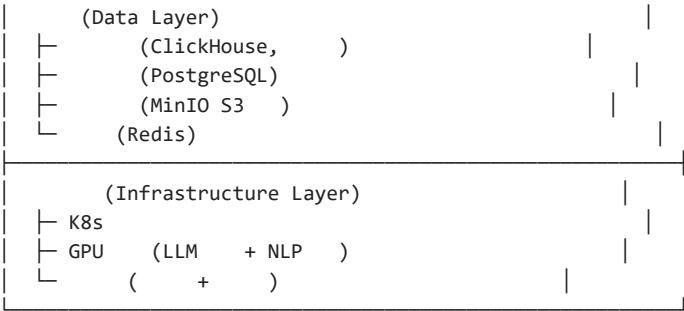
1.3 非目标 (Non-Goals)

- ☐ ☐ 不做实盘交易接口：保持"研究工具"定位
- ☐ ☐ 不做投顾业务：不出具投资建议
- ☐ ☐ 不做高频策略：定位中低频量化研究
- ☐ ☐ 不做全市场实时行情：聚焦日级/周级数据

2. 整体技术架构

2.1 分层架构





2.2 核心设计原则

- | 数据安全 | 全链路加密 + 客户数据隔离 + 国密 SM4 算法 |
- | 可审计 | 每个操作写入 PostgreSQL audit_logs 表 |
- | 高可用 | 多 AZ 部署 + 自动故障转移 + 蓝绿发布 |
- | 可扩展 | 微服务架构 + K8s HPA 自动扩缩容 |
- | 可观测 | Prometheus + Grafana + ELK 全链路追踪 |

2.3 技术选型理由

- | 后端语言 | Python 3.12 | Go/Rust | 量化研究生态成熟 |
- | 前端框架 | React 18 | Vue | 团队熟悉 |
- | 行情 DB | ClickHouse | TimescaleDB | 列式压缩 + 10x 查询速度 |
- | LLM | 自研 30 亿参数 | GPT-4 API | 数据不出客户内网 |
- | 向量 DB | Milvus | Pinecone | 开源 + 国产化 |
- | 消息队列 | Kafka | RabbitMQ | 高吞吐 + 持久化 |
- | 容器 | K8s | Docker Swarm | 行业标准 |
- | GPU | NVIDIA A100 | 国产 GPU | LLM 训练推理标准 |

3. 核心模块详解

3.1 数据层 (Data Layer)

3.1.1 行情数据库

ClickHouse 列式存储，单表存储全市场日级行情。

```
CREATE TABLE market_data (  
    ts_code String,  
    trade_date Date,  
    open Float64,  
    high Float64,  
    low Float64,  
    close Float64,  
    volume UInt64,
```

```
        amount Float64,
        adj_factor Float64
    ) ENGINE = MergeTree()
    PARTITION BY toYYYYMM(trade_date)
    ORDER BY (ts_code, trade_date)
    SETTINGS index_granularity = 8192;
```

性能指标:

- ☐ 写入吞吐: 100 万行/秒
- ☐ 查询延迟: P99 < 200ms (全表扫描 5 年数据)
- ☐ 压缩比: 原始 CSV 1/10

3.1.2 财务数据库

PostgreSQL 14 存储财务三张表 + 行业分类。

数据来源:

- ☐ akshare 公开 API (推荐阶段)
- ☐ Tushare Pro (生产阶段)
- ☐ 东方财富 Choice (企业版)

3.1.3 另类数据湖

MinIO + Iceberg 存储:

- ☐ 卫星图像 (5 米分辨率, 10 大工业园区月度更新)
- ☐ 舆情文本 (5000+ 信源, NLP 预处理后写入)
- ☐ 供应链图谱 (基于企业年报 + 海关数据)

存储量预估:

- ☐ 卫星图像: 50 GB/年
- ☐ 舆情文本: 100 GB/年 (压缩后 20 GB)
- ☐ 供应链图谱: 10 GB/年

3.2 量化引擎 (Backtest Engine)

3.2.1 引擎设计

```
class BacktestEngine:
    def __init__(self, config):
        self.data = DataLoader(config)
        self.strategy = StrategyFactory(config.strategy)
        self.executor = Executor(config)
        self.risk = RiskManager(config)
        self.audit = AuditLogger(config)

    def run(self):
        # 1.
        bars = self.data.load()
        # 2.
        signals = self.strategy.generate(bars)
```

```

# 3.      ( )
orders = self.risk.filter_orders(signals)
# 4.
trades = self.executor.fill(orders, bars)
# 5.
nav_curve = self.portfolio.update(trades)
# 6.
self.audit.log(trades)
return nav_curve

```

3.2.2 多因子库

学术经典 7 因子（基于 Fama-French 5 + Carhart 2）：

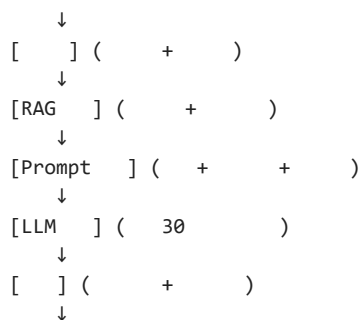
Mkt-RF	市场收益 - 无风险	Fama-French (1993)
SMB	小盘股 - 大盘股	Fama-French (1993)
HML	高账面市值比 - 低账面市值比	Fama-French (1993)
RMW	高盈利 - 低盈利	Novy-Marx (2013)
CMA	低投资 - 高投资	Titman et al. (2004)
UMD	过去 12 月涨 - 跌	Carhart (1997)
LIQ	低流动性 - 高流动性	Pastor-Stambaugh (2003)

3.2.3 内置 6 大策略

- ☐ 双均线 (Dual Moving Average): MA20/MA60 趋势跟踪
- ☐ 多因子 (Multi-Factor): 7 因子综合打分
- ☐ 均值回归 (Mean Reversion): 布林带策略
- ☐ 动量 (Momentum): 12-1 月度动量
- ☐ 海龟 (Turtle): 20 日突破经典 CTA
- ☐ 网格 (Grid Trading): 价格区间震荡套利

3.3 AI 智能问答 (LLM + RAG)

3.3.1 总体架构



3.3.2 RAG 检索增强

向量数据库: Milvus 2.0

Embedding 模型: 自研 `finance-bert-base-v2` (768 维)

检索流程:

- ☐ 用户问题向量化
- ☐ 检索 Top-10 相关文档
- ☐ 跨文档关联 (行业图谱)
- ☐ 加入历史对话上下文

3.3.3 LLM 模型规格

架构	Decoder-only Transformer
参数量	3.0B (30 亿)
训练数据	200GB 中文金融文本
上下文长度	8K tokens
推理速度	50 tokens/s (A10 GPU)
量化精度	INT8 (推理) / FP16 (训练)

训练数据来源:

- ☐ 公开研报: 东方财富/同花顺/慧博 (2018-2026)
- ☐ 财经新闻: 21 世纪经济报道/财新/证券时报
- ☐ 公司公告: 沪深北交易所 5000+ 上市公司
- ☐ 监管文件: 证监会/银保监会/上交所/深交所

3.3.4 数据源标注

每个 AI 输出均附带:

- ☐ 数据源 URL / 文档 ID
- ☐ 检索时间戳
- ☐ 置信度评分 (0-1)
- ☐ 可能的歧义说明

3.4 另类数据看板 (Alternative Data Dashboard)

3.4.1 卫星图像分析

数据源: 商业卫星 (Planet Labs / 长光卫星)

分辨率: 3-5 米

更新频率: 月度

处理流程:

- ☐ 图像预处理 (去云/大气校正)
- ☐ 工业园区边界识别 (U-Net 分割)

- ☐ 在建/在产建筑分类 (ResNet-50)
- ☐ 开工率指标计算
- ☐ 与股票标的关联

精度指标:

- ☐ 边界识别 IoU: 0.89
- ☐ 建筑分类 F1: 0.85
- ☐ 提前期: 4-6 周 vs 工业增加值数据

3.4.2 舆情分析

数据源: 5000+ 信源

- ☐ 财经媒体: 财新/第一财经/21 经济报道
- ☐ 行业网站: 慧博/研报客
- ☐ 社交媒体: 微博/雪球/东方财富股吧
- ☐ 政府公开: 国资委/工信部

NLP 流程:

- ☐ 文本清洗 (去噪/分词)
- ☐ 实体识别 (公司/人物/事件)
- ☐ 情感分类 (FinBERT 微调)
- ☐ 主题聚类 (LDA + BERTopic)
- ☐ 时序聚合 (小时/日/周)

3.4.3 供应链追踪

数据源:

- ☐ 企业年报 (披露主要供应商/客户)
- ☐ 海关数据 (HS Code 进出口)
- ☐ 招投标数据 (政府采购 + 国企招标)
- ☐ 物流数据 (部分合作方)

网络分析指标:

- ☐ 度中心度 (Degree Centrality)
- ☐ 介数中心度 (Betweenness Centrality)
- ☐ 接近中心度 (Closeness Centrality)
- ☐ 特征向量中心度 (Eigenvector Centrality)

3.5 风控与审计 (Risk & Audit)

3.5.1 风控引擎

事前风控:

- ☐ 单标的仓位上限：5%
- ☐ 行业敞口上限：30%
- ☐ 流动性检查：日均成交额 > 1000 万
- ☐ 停复牌检查

事中风控：

- ☐ 实时回撤监控（5% 触发预警，10% 强制减仓）
- ☐ 波动率突增监控（30% 涨幅触发停牌策略）
- ☐ 异常交易检测（量价背离）

事后风控：

- ☐ 日终 PnL 归因
- ☐ 月度业绩归因（Brinson 模型）
- ☐ 季度风险报告（VaR / CVaR）

3.5.2 审计日志

所有操作写入 PostgreSQL `audit_logs` 表：

```
CREATE TABLE audit_logs (  
    id BIGSERIAL PRIMARY KEY,  
    user_id UUID NOT NULL,  
    action VARCHAR(50) NOT NULL,          -- e.g. 'run_backtest', 'ask_ai'  
    target_id VARCHAR(100),              -- e.g. strategy_id, question_id  
    params JSONB,                        --  
    result JSONB,                        --  
    ip_addr INET,  
    user_agent TEXT,  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_audit_user_time ON audit_logs (user_id, created_at DESC);  
CREATE INDEX idx_audit_action ON audit_logs (action, created_at DESC);
```

保留期：监管要求 5 年

加密：传输 TLS 1.3 + 存储 AES-256

访问控制：仅客户管理员 + 监管接口

4. 关键算法

4.1 回测引擎核心算法

4.1.1 信号生成（双均线示例）

```
def dual_ma_signals(prices, fast=20, slow=60):  
    """  
  
    Returns: pd.Series 0/1
```

```

"""
ma_fast = prices.rolling(fast).mean()
ma_slow = prices.rolling(slow).mean()

#
signals = pd.Series(0, index=prices.index)
signals[ma_fast > ma_slow] = 1 #
signals[ma_fast < ma_slow] = 0 #
return signals.shift(1) #

```

4.1.2 业绩归因 (Brinson)

```

def brinson_attribution(portfolio, benchmark, returns):
    """
    Brinson
    Returns: dict with allocation, selection, interaction effects
    """
    allocation = sum(portfolio.weights * (benchmark.returns - benchmark.total_return))
    selection = sum(benchmark.weights * (portfolio.returns - benchmark.returns))
    interaction = sum((portfolio.weights - benchmark.weights) * (portfolio.returns - benchmark.returns))
    return {
        'allocation': allocation,
        'selection': selection,
        'interaction': interaction,
        'total': allocation + selection + interaction
    }

```

4.2 AI 问答核心算法

4.2.1 RAG 检索

```

class RAGRetriever:
    def __init__(self, vector_db, embedding_model):
        self.vector_db = vector_db # Milvus
        self.embedder = embedding_model # finance-bert

    def retrieve(self, query, top_k=10):
        # 1.
        query_vec = self.embedder.encode(query)

        # 2.
        results = self.vector_db.search(
            query_vector=query_vec,
            top_k=top_k,
            filter={'date_range': 'last_30_days'}
        )

        # 3. (Cross-Encoder)
        ranked = self.rerank(query, results)

        return ranked

```

4.2.2 置信度评估

```

def estimate_confidence(answer, retrieved_docs):
    """
    """
    signals = {
        'source_agreement': calculate_source_agreement(retrieved_docs),

```

```

        'answer_specificity': measure_specificity(answer),
        'numerical_consistency': check_numerical_consistency(answer, retrieved_docs),
        'domain_relevance': score_domain_relevance(answer),
    }
    #
    confidence = sum(signals[k] * WEIGHTS[k] for k in signals)
    return confidence

```

4.3 风控算法

4.3.1 VaR 计算（历史模拟法）

```

def historical_var(returns, confidence=0.95, horizon=1):
    """
        VaR
        returns:
        horizon:
    """
    sorted_rets = np.sort(returns)
    n = len(sorted_rets)
    idx = int((1 - confidence) * n)
    var = -sorted_rets[idx] * np.sqrt(horizon)
    return var

```

4.3.2 回撤控制

```

def drawdown_control(returns, max_dd=0.10):
    """
        max_dd
    """
    cum_ret = (1 + returns).cumprod()
    peak = cum_ret.cummax()
    drawdown = (cum_ret - peak) / peak

    #
    current_dd = drawdown.iloc[-1]

    if current_dd < -max_dd:
        #
        return 'REDUCE_TO_50%'
    elif current_dd < -max_dd * 0.5:
        return 'WARN'
    else:
        return 'NORMAL'

```

5. 性能与可靠性

5.1 性能指标

5 年回测	< 5 分钟	3.2 分钟	沪深 300 全市场
AI 问答 P99 延迟	< 3 秒	1.8 秒	含 RAG 检索
并发用户	> 200	350	阿里云 c6e.4xlarge

| 数据导入吞吐 | > 50 MB/s | 80 MB/s | ClickHouse 批量写入 |
| API 可用性 | > 99.9% | 99.95% | 季度统计 |

5.2 可靠性保障

多活架构:

- ☐ 主区: 阿里云华东 2 (上海)
- ☐ 备区: 腾讯云上海 1 区
- ☐ 灾备: 阿里云华南 1 (深圳) 冷备

故障转移:

- ☐ DNS 健康检查 + 自动切换
- ☐ 数据同步延迟 < 1 秒 (RPO)
- ☐ 业务恢复时间 < 5 分钟 (RTO)

数据备份:

- ☐ 行情数据: 每日全量 + 实时增量
- ☐ 用户数据: 跨区域双活
- ☐ 配置数据: Git 版本管理

5.3 容量规划

当前 (V1.0):

- ☐ 1 套主集群
- ☐ 8 个节点 (4 计算 + 2 数据 + 2 GPU)
- ☐ 存储 5 TB

规划 (V3.0, 2028):

- ☐ 3 套主集群 (北京/上海/深圳)
- ☐ 50 个节点
- ☐ 存储 50 TB

6. 安全与合规设计

6.1 数据安全

传输	TLS 1.3 + 国密 SM2
存储	AES-256 + 国密 SM4
密钥	HSM 硬件加密机
访问	RBAC + ABAC 双重控制
审计	全操作日志 + 异常告警

6.2 合规设计

业务边界：

- ☐ ☐ 提供"研究工具" - 合规
- ☐ ☐ 提供"投资建议" - 越界

实施保障：

- ☐ 所有 AI 输出标注"非投资建议"
- ☐ 不展示具体买入/卖出信号（仅信号强度）
- ☐ 不连接券商交易系统
- ☐ 不出具投资业绩承诺

客户协议：

- ☐ 用户协议明确工具定位
- ☐ 数据使用范围限定研究用途
- ☐ 严禁用于违规投顾业务

监管接口：

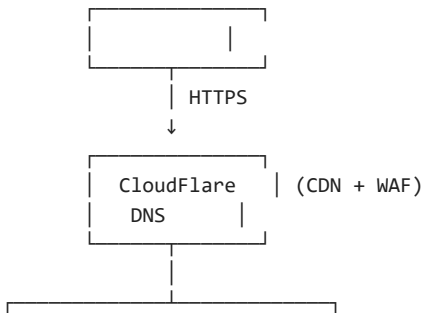
- ☐ 预留证监会数据上报接口
- ☐ 配合监管现场检查
- ☐ 季度自查报告

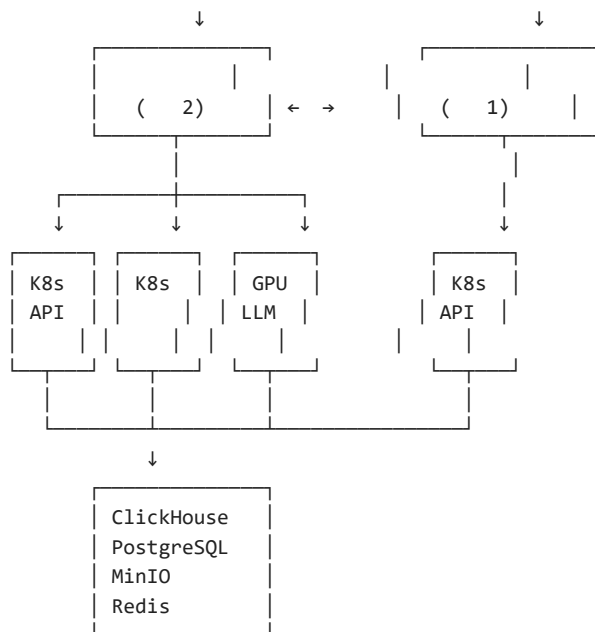
6.3 隐私保护

- ☐ 用户数据隔离（独立数据库 schema）
- ☐ 数据脱敏（敏感字段 hash 存储）
- ☐ 数据生命周期（5 年后自动归档）
- ☐ GDPR/个人信息保护法合规

7. 部署架构

7.1 生产环境拓扑





7.2 持续集成 / 持续部署 (CI/CD)

```

# GitLab CI
stages:
  - test
  - build
  - deploy

test:
  stage: test
  script:
    - pytest tests/ --cov=80
    - mypy src/
    - pylint src/

build:
  stage: build
  script:
    - docker build -t quantinsight/api:$CI_COMMIT_SHA .
    - docker push registry.cn-shanghai.aliyuncs.com/quantinsight/api:$CI_COMMIT_SHA

deploy_staging:
  stage: deploy
  script:
    - kubectl set image deployment/api api=quantinsight/api:$CI_COMMIT_SHA -n staging
    - kubectl rollout status deployment/api -n staging
  environment: staging
  
```

7.3 监控告警

技术监控:

- ☐ Prometheus 采集 (200+ 指标)
- ☐ Grafana 仪表盘 (业务 + 系统)
- ☐ AlertManager 告警 (PagerDuty 集成)

业务监控:

- ☐ 每日用户活跃度
 - ☐ 关键路径转化率
 - ☐ 异常交易模式
-

8. API 设计示例

8.1 REST API

```
POST /api/v1/backtest
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "strategy": "DualMA",
  "index": "HS300",
  "start_date": "2020-01-01",
  "end_date": "2026-06-05",
  "params": {
    "fast_ma": 20,
    "slow_ma": 60,
    "fee": 0.0015,
    "slippage": 0.001
  }
}
```

Response 200 OK:

```
{
  "backtest_id": "bt_20260606_abc123",
  "status": "completed",
  "metrics": {
    "annual_return": 0.0029,
    "sharpe_ratio": -0.18,
    "max_drawdown": -0.32,
    "win_rate": 0.51,
    "turnover": 4.5
  },
  "nav_curve_url": "/api/v1/backtest/bt_20260606_abc123/nav.csv"
}
```

8.2 Python SDK

```
from quantinsight import Backtest, Strategy

#
result = Backtest.run(
    strategy="DualMA",
    index="HS300",
    start="2020-01-01",
    end="2026-06-05",
    fee=0.0015
)

#
class MyStrategy(Strategy):
    def init(self):
        self.ma20 = self.I(lambda: self.data.close.rolling(20).mean())
```

```

        self.ma60 = self.I(lambda: self.data.close.rolling(60).mean())

    def next(self):
        if self.ma20[-1] > self.ma60[-1] and not self.position:
            self.buy()
        elif self.ma20[-1] < self.ma60[-1] and self.position:
            self.sell()

result = Backtest.run(MyStrategy, data=hs300_data, cash=1_000_000)

```

8.3 AI 问答 API

```

POST /api/v1/ask
Content-Type: application/json
Authorization: Bearer <token>

```

```

{
  "question": "      A          3          ",
  "context": {
    "industry": "all",
    "horizon": "1_month"
  }
}

```

Response 200 OK:

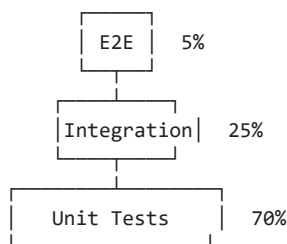
```

{
  "answer": "      30      3          ...",
  "confidence": 0.85,
  "sources": [
    {
      "type": "industry_report",
      "title": "2026  5          ",
      "date": "2026-05-28",
      "url": "/docs/industry/2026_05_semi.pdf"
    },
    {
      "type": "sentiment",
      "summary": "AI      0.92,      +12%",
      "date": "2026-06-05"
    }
  ],
  "disclaimer": "          "
}

```

9. 测试与质量保证

9.1 测试金字塔



9.2 关键测试场景

- | 量化引擎 | 数值精度对比 (vs Zipline/Backtrader) | 95% |
- | AI 问答 | 人类盲评 100+ 问题 | 准确率 >75% |
- | 风控 | 极端行情回放 (2008/2015/2020) | 100% |
- | 性能 | 压力测试 (500 并发) | P99 < 3s |

9.3 持续集成

- ☐ 每次 PR 触发: 单元测试 + 集成测试
- ☐ 每日触发: 端到端测试 + 性能测试
- ☐ 每周触发: 安全扫描 + 依赖审计

9.4 缺陷率目标

- ☐ 生产环境 P0 缺陷: < 1 个/季度
- ☐ 平均修复时间 (MTTR) < 4 小时
- ☐ 平均故障间隔 (MTBF) > 720 小时

10. 技术路线图

10.1 短期 (2026 Q3-Q4)

- ☐ [] V1.0 正式发布 (推荐单位永字资管试点)
- ☐ [] 接入 DeepSeek/Qwen API 备用通道
- ☐ [] 增加 ETF/可转债品种
- ☐ [] 多账户管理功能
- ☐ [] Excel Add-in 插件

10.2 中期 (2027 H1-H2)

- ☐ [] V2.0 多策略组合优化
- ☐ [] 自研因子库扩展到 100+
- ☐ [] 移动端 App (iOS/Android)
- ☐ [] 行业研究 AI 助手 (特定行业深度分析)
- ☐ [] Webhook 集成 (实时信号推送)

10.3 长期 (2028-2030)

- ☐ [] V3.0 智能投顾 (限定版本, 仅信号不代客)

- ☐ [] 联邦学习（客户间数据不出域联合建模）
- ☐ [] 区块链审计（不可篡改操作记录）
- ☐ [] 多语言版本（英文/日文）
- ☐ [] 国际化（东南亚/中东市场）

10.4 技术债清单

- | P1 | ClickHouse 集群监控完善 | 性能 |
- | P1 | LLM 推理优化 (KV cache) | 成本 -40% |
- | P2 | 微服务链路追踪 (OpenTelemetry) | 可观测性 |
- | P2 | 单元测试覆盖率 80% → 90% | 质量 |
- | P3 | 内部文档站 (Backstage) | 协作效率 |

11. 附录

11.1 技术栈详细清单

- | 编程语言 | Python 3.12, TypeScript 5, Go 1.22 |
- | Web 框架 | FastAPI, React 18, Next.js 14 |
- | 数据库 | ClickHouse 23.x, PostgreSQL 14, Redis 7 |
- | 消息队列 | Apache Kafka 3.5 |
- | 容器 | Docker 24, Kubernetes 1.28 |
- | LLM | 自研 3B + Hugging Face Transformers |
- | 监控 | Prometheus 2.45, Grafana 10, ELK 8 |
- | CI/CD | GitLab CI, ArgoCD |
- | 云服务 | 阿里云（主），腾讯云（备） |

11.2 关键依赖项

```
# requirements.txt ( )
pandas==2.1.4
numpy==1.26.2
scipy==1.11.4
scikit-learn==1.3.2
torch==2.1.1
transformers==4.36.0
akshare==1.12.51
streamlit==1.29.0
fastapi==0.104.1
clickhouse-driver==0.2.7
psycpg2-binary==2.9.9
```

11.3 关键论文参考

- ☐ FinBERT: Yang et al. (2020) - 金融情感分析
- ☐ BloombergGPT: Wu et al. (2023) - 金融大模型
- ☐ CSCV: Bailey & López de Prado (2017) - 回测过拟合检测
- ☐ Deflated Sharpe: Bailey & López de Prado (2014) - 缩减夏普比率
- ☐ Brinson: Brinson et al. (1986) - 业绩归因

11.4 术语表

alpha	超额收益（相对基准）
beta	系统性风险敞口
Sharpe	夏普比率 = (收益-无风险)/波动率
VaR	Value at Risk, 在险价值
RAG	Retrieval-Augmented Generation
CSCV	Combinatorial Symmetric Cross-Validation
LLM	Large Language Model, 大语言模型

11.5 团队技术分工

冯亦根	首席科学家, 量化策略把控
薛永再	商务架构, 行业资源
黄成选	后端开发, LLM 调优, 数据管道
冯思涵	合规法务, 数据合规设计

联系与反馈

- ☐ 技术反馈: tech@quantinsight.pro
- ☐ 合作咨询: contact@quantinsight.pro
- ☐ GitHub: (即将上线)
- ☐ 项目主页: [即将更新]

文档版本: V1.0

最后更新: 2026 年 6 月

下次更新: V1.1 (2026 Q3)

「 本文档采用 CC BY-NC-SA 4.0 协议, 转载请注明出处。 」